



نظري

كلية الهندسة المعلوماتية

السنة الثانية

Java OOP (مراجعة)

د. غاندي هسام



RB Informatics; 9/10/2025

برمجة ٣

بسم الله الرحمن الرحيم مرحبا زملائي في مادة البرمجة 3 التي سنتابع بها بلغة JAVA وسنتعلم مفاهيم جديدة وممتعة تساعدنا على تطوير مهارتنا البرمجية فلنبدأ رحلتنا بحماس ٣ >.

بدايةً وكما نذكر مما تعلمنا في البرمجة 2 أن الجافا لغة قائمة على ال OOP (البرمجة كائنية التوجه) مما يساعد على تنظيم الكود باستخدام الوحدات المنطقية الاساسية (Classes and Objects) وتجعل الكود قابل للتعديل وإعادة الاستخدام والتوسع والتطوير بالإضافة الى الحفاظ على البيانات من الوصول العشوائي إضافة الى انها توفر هيكلية منطقية وواضحة حيث يتم تقسيم البرنامج إلى Classes، كل منها مسؤول عن وظيفة معينة. هذا يجعل الكود أسهل في:

- الفهم
- الصيانة
- تصحيح الأخطاء (Debugging)

ملاحظة :

أهم قاعدة في البرمجة هي قاعدة ال DRY

" Don't Repeat your self "

وال java OOP تساعدنا بتحقيقها

إضافة ارسيزية 😊 :

- ال oop توفر العديد من الميزات مثل :
- 1. Maintainability - سهولة الصيانة
- 2. Scalability - قابلية التوسع
- 3. Flexibility - المرونة
- 4. Readability & Understandability - قابلية القراءة والفهم

1. (Class) في java

هو قالب (نموذج أولي) يُنشئه المبرمج لتحديد شكل الكائنات (Objects) التي سيتم إنشاؤها لاحقاً. يمثل Class مجموعة من الخصائص (Properties) والدوال (Methods) المشتركة بين جميع الكائنات من نفس النوع.

من خلال استخدام الـ classes، يمكن إنشاء عدة كائنات لها نفس السلوك دون الحاجة إلى إعادة كتابة الكود لكل كائن على حدة، مما يعزز قابلية إعادة الاستخدام والتنظيم.

تعريف الـ Class في Java

يتكون تعريفه عادةً من العناصر التالية بالترتيب :

(Modifiers) : مثل public أو بدون تعديل (افتراضي)، وتحدد مستوى الوصول لهذا الـ Class.

(Class Name) : يُفضل أن يبدأ بحرف كبير حسب العرف البرمجي.

(Body) : يُكتب بين الأقواس {} ويحتوي على المتغيرات والدوال التي تمثل سلوك الكائنات.

2. Object في Java

الكائن هو الوحدة الأساسية في البرمجة الكائنية ، ويُستخدم لتمثيل الكيانات الواقعية داخل البرنامج. يمكن أن يكون الكائن سيارة 🚗 ، طالب 🧑 ، حساب بنكي 🏦 ، أو حتى روبوت 🤖 .. أي شيء له خصائص وسلوك. في برامج Java ، يتم إنشاء العديد من الكائنات التي تتفاعل فيما بينها عن طريق استدعاء الدوال (Methods) .

مكونات الكائن الأساسية:

كل كائن يتكون من أربع عناصر رئيسية:

الحالة (State) : تمثلها الخصائص (Attributes) مثل اللون، السرعة، الاسم. تعكس صفات الكائن.

السلوك (Behavior) : تمثله الدوال (Methods) مثل speak(), move() يحدد كيف يتصرف الكائن.

الهوية (Identity) : اسم فريد للكائن يميزه عن غيره، مثل carA، robot1 يسمح بالتفاعل بين الكائنات.

الدالة (Method) : مجموعة أوامر تنفذ مهمة محددة وتعيد نتيجة. تُستخدم للتفاعل مع الكائن أو تغييره.



"Life is *short* , Follow what *sparks* your soul ✨".

مثال ب code:

```

public class Employee {
    // Instance variables (non-static)
    private String name;
    private double salary;
    // Construction
    public Employee(String name, double salary) {
        this.name = name;
        this.salary = salary;
    }
    // getters method
    public String getName() { return name; }
    public double getSalary() { return salary; }
    // setters method
    public void setName(String name) { this.name = name; }
    public void setSalary(double salary) { this.salary = salary; }
    // Instance method
    public void displayDetails() {
        System.out.println("Employee: " + name);
        System.out.println("Salary: " + salary);
    }
    public static void main(String[] args) {
        Employee emp = new Employee("Geek", 10000.0f);
        emp.displayDetails(); }
}

```

3. Abstraction في Java

التجريد هو أحد الركائز الأساسية في البرمجة الكائنية، ويُقصد به إخفاء التفاصيل الداخلية لتنفيذ الكود والتركيز فقط على الوظائف الأساسية التي يحتاجها المستخدم أو المبرمج.

بعبارة أخرى: التجريد يسمح لنا بأن نسأل: "ماذا يفعل هذا الكائن؟" دون الحاجة لمعرفة كيف يفعل ذلك.

فائدة أربيسيزية ☺ : الفكرة الأساسية هي أن :

- لمستخدم يتعامل مع واجهة بسيطة توضح الوظائف المتاحة.
- التفاصيل المعقدة التي تحدث خلف الكواليس يتم إخفاؤها.
- هذا يسهل استخدام الكائنات ويقلل من التعقيد.
- **مثال واقعي:** عند استخدام دالة `print()` ، أنت تعرف أنها تطبع على الشاشة، لكنك لا تحتاج لمعرفة كيف تتعامل مع نظام التشغيل أو تعريفات الطباعة.

كيف ينفذ التجريد؟

هناك طريقتين لتحقيقه:

1. الواجهات (Interfaces) :

تحقق تجريد كامل بنسبة 100٪. تحتوي فقط على تعريفات للدوال بدون تنفيذ.

2. الأصناف المجردة (Abstract Classes) :

تحقق تجريد جزئي. يمكن أن تحتوي على دوال معرفة وأخرى غير معرفة.

مثال بـ code :

```

abstract class Vehicle {
    // Abstract methods (what it can do)
    abstract void accelerate();
    abstract void brake();
    // Concrete method (common to all vehicles)
    void startEngine() {
        System.out.println("Engine started...");
    }
}

// Concrete implementation (hidden details)
class Car extends Vehicle {
    @Override
    void accelerate() {
        System.out.println("Car: Pressing gas pedal...");
        // Hidden complex logic: fuel injection, gear shifting, etc.
    }
    @Override
    void brake() {
        System.out.println("Car: Applying brakes...");
        // Hidden Logic: hydraulic pressure, brake pads, etc.
    }
}

public class Main {
    public static void main(String[] args) {
        Vehicle myCar = new Car();
        myCar.startEngine();
        myCar.accelerate();
        myCar.brake();
    }
}

```



Java Encapsulation في

هو عملية دمج البيانات والدوال التي تتعامل معها داخل وحدة واحدة، وغالبًا ما تكون هذه الوحدة هي ال (Class) يُعتبر التغليف آلية تربط الكود بالبيانات بطريقة تحافظ على خصوصية المعلومات وتمنع الوصول غير المصرح به.

■ أي:

- يتم إخفاء المتغيرات داخل الصنف عن باقي أجزاء البرنامج.
- لا يمكن الوصول إلى هذه المتغيرات إلا من خلال دوال معرفة داخل نفس الصنف.
- هذا المفهوم يُعرف أيضًا باسم إخفاء البيانات (Data Hiding) ، وغالبًا ما يُستخدم المصطلحان بالتبادل.

■ كيف نحقق التغليف في Java ؟

- نجعل المتغيرات خاصة (private) داخل ال Class
- نكتب دوال عامة (public) تسمح بالوصول إلى هذه المتغيرات بطريقة مضبوط، وهما :
- دالة get لاسترجاع القيمة.
- دالة set لتعديل القيمة.

مثال ب code :

```
class Employee {  
    // Private fields (encapsulated data)  
    private int id;  
    private String name;  
  
    // Setter methods  
    public void setId(int id) {  
        this.id = id;  
    }  
  
    public void setName(String name) {  
        this.name = name;  
    }  
  
    // Getter methods  
    public int getId() {  
        return id;  
    }  
}
```

```
public String getName() {  
    return name;  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Employee emp = new Employee();  
  
        // Using setters  
        emp.setId(101);  
        emp.setName("Geek");  
  
        // Using getters  
        System.out.println("Employee ID: " + emp.getId());  
        System.out.println("Employee Name: " + emp.getName());  
    }  
}
```

Output:

Employee ID: 101

Employee Name: Geek

5 . Inheritance في Java

الوراثة هي أحد الأعمدة الأساسية في البرمجة الكائنية، وتستخدم لتمكين صنف (Class) من اكتساب الخصائص والدوال من صنف آخر. هذا يسمح بإعادة استخدام الكود وتوسيع الوظائف دون الحاجة إلى إعادة كتابة كل شيء من الصفر.

في Java ، يتم تطبيق الوراثة باستخدام الكلمة المفتاحية `extends` .

■ علاقة "is-a" :

الوراثة تُنشئ علاقة "is-a" بين ال Classes أي:
إذا كان Dog يرث من Animal، فهذا يعني أن "الكلب هو حيوان."
الأصناف المشتقة (Derived Classes) ترث الخصائص والسلوك من الصنف الأساسي (Base Class) ،
ويمكنها أيضاً إضافة أو تعديل وظائفها الخاصة.

■ Superclas:

■ هو الصنف الذي يحتوي على الخصائص والدوال التي يتم توريثها. يُعرف بالصنف الأساسي أو الأب، ويُستخدم كنقطة انطلاق لبناء أصناف أكثر تخصصاً.

■ مثال Animal : هو صنف عام يحتوي على دوال مثل eat() و sleep().

■ Subclass:

■ هو الصنف الذي يرث من صنف آخر (يُعرف بـ Class الابن). يمكنه استخدام الخصائص والدوال من الـ Class الأب، بالإضافة إلى تعريف خصائص ودوال خاصة به.

■ مثال Dog و Cat يرثان من Animal، ويمكنهما إضافة دوال مثل bark() أو meow().

■ Reusability:

■ الوراثة تدعم مبدأ إعادة الاستخدام، حيث يمكننا إنشاء صنف جديد بناءً على صنف موجود مسبقاً يحتوي على جزء من الكود المطلوب. هذا يوفر الوقت ويقلل التكرار.

■ مثال عملي:

■ إذا كان لدينا صنف Vehicle يحتوي على دالة startEngine() ، يمكن لصنف Car أن يرث منه ويستخدم نفس الدالة دون إعادة كتابتها.

■ تذكرة:

■ Overriding:

■ هو مفهوم يسمح للصنف الابن بإعادة تعريف دالة موجودة في الصنف الأب بنفس الاسم والتوقيع، لكن بتنفيذ مختلف.

مثال بـ code :

```
class Animal {
    void eat() {
        System.out.println("Animal is eating...");
    }
    void sleep() {
        System.out.println("Animal is sleeping...");
    }
}
class Dog extends Animal {
    // Subclass (Child) - Inherits from Animal
    void bark() {
        System.out.println("Dog is barking!");
    }
}
```



```
public class Main {
    public static void main (String[] args) {
        Dog myDog = new Dog();
        // Inherited methods from Animal
        myDog.eat();
        myDog.sleep();
        // Child class method
        myDog.bark();
    }
}
```

Polymorphism.6

تعني **تعدد الأشكال**، أي أن الكيان الواحد يمكن أن يتخذ عدة أشكال مختلفة. في البرمجة الكائنية باستخدام Java ، يُستخدم هذا المفهوم للسماح لنفس الدالة أو الكائن بأن يتصرف بطرق مختلفة حسب الحاجة والموضع وخاصةً أثناء وقت التشغيل.

■ أنواع التعددية الشكلية:

■ Method Overloading:

■ يُعرف أيضاً باسم **التعددية الاشكال في وقت الترجمة (Compile-Time Polymorphism)**.

■ يحدث عندما تحتوي نفس ال class على عدة دوال بنفس الاسم ولكن ب Parameters مختلفة (عدد أو نوع المعاملات).

■ يمكن أن تختلف أنواع الإرجاع، لكن الأهم هو اختلاف ال Parameters

■ Method Overriding:

■ يُعرف أيضاً باسم **التعددية الأشكال في وقت التشغيل (Run-Time Polymorphism)**.

■ يحدث عندما يقوم ال class الابن بإعادة تعريف دالة موجودة في ال class الأب بنفس الاسم والتوقيع.

■ التنفيذ الفعلي يتم تحديده حسب نوع الكائن المستخدم أثناء التشغيل.

تذكرة :

ما المقصود ب نوع الكائن المستخدم أثناء التشغيل؟

أي نوع الكائن الحقيقي وليس نوع المتغير الذي يشير اليه عندما نكتب

Animal a1 = new Dog();

أي أن المتغير a1 هو من نوع Animal ولكنه يشير الى كائن من نوع Dog

```
// Parent Class
class Parent {
    // Overloaded method (compile-time polymorphism)
    public void func(int a) {
        System.out.println("Parent.func(int): " + a);
    }
}

// Overloaded method (same name, different parameter)
class Parent {
    public void func(double a) {
        System.out.println("Parent.func(double): " + a);
    }
}

// Child Class
class Child extends Parent {
    // Method Overriding
    @Override
    public void func(int a) {
        System.out.println("Child.func(int): " + a);
    }
}

public class Main {
    public static void main(String[] args) {
        Parent parent = new Parent();
        Child child = new Child();
        // Dynamic dispatch
        Parent polymorphObj = new Child();
        // Method Overloading
        parent.func(10);
        parent.func(20.0);
        // Method Overriding
        child.func(20);
        polymorphObj.func(30);
    }
}
```



**Output:**

```
Parent.func(int): 10
Parent.func(double): 20.0
Child.func(int): 20
Child.func(int): 30
```

عيوب البرمجة الكائنية (Object-Oriented Programming)

1. صعوبة الفهم للمبتدئين:
مفاهيمها تحتاج وقتًا وجهدًا لفهم كيف تتفاعل معًا لبناء نظام متكامل.
مثال: طالب جديد قد يواجه صعوبة في فهم الفرق بين
`Dog d = new Dog();` و `Animal a = new Dog();` وبين
2. ثقل في البرامج الصغيرة:
عند كتابة برامج بسيطة، قد تبدو بنية OOP زائدة عن الحاجة .
3. صعوبة تتبع الأخطاء:
نظرًا لأن الكود في OOP مقسم إلى طبقات وأصناف متعددة، فإن تتبع الأخطاء قد يستغرق وقتًا أطول
4. استهلاك أكبر للذاكرة:
OOP تعتمد على إنشاء عدد كبير من الكائنات، وكل كائن يحتفظ بحالته وسلوكه الخاص. هذا قد يؤدي إلى زيادة استهلاك الذاكرة مقارنة بالبرمجة الإجرائية التي تستخدم دوال بسيطة ومتغيرات عامة.

Java Says

your Tears Are Delicious :(

